ASSIGNMENT 2 – CO2

TASK 02

Q2 – When Is ArrayList an Appropriate Engineering Choice?

Course: CSA0903 – Programming in Java
Problem Context: Railway e-Ticket Generation

Name: Sowmiya. D

Register Number: 192511063

1. Aim

To compare the performance of ArrayList, LinkedList and ArrayDeque for different playlist operations and identify the most suitable data structure based on insertion, deletion, searching and iteration requirements.

2. Prediction

- **ArrayList: Provides fast random access and efficient insertion at the end. Inserting or removing elements at the front or middle requires shifting elements.**
- **LinkedList: Supports efficient insertion and deletion at both ends. However, accessing an element by index requires traversal.**
- **ArrayDeque: Provides efficient insertion and removal at both ends. It is suitable for implementing an Up Next queue but does not support indexed access.**

3. Experimental Design

Three playlist sizes are selected:

Experiment	**Number of songs**
Test 1	1,000
Test 2	10,000
Test 3	50,000

The following operations are compared:

1. Adding songs at the end.
2. Inserting songs at the front.
3. Inserting songs in the middle.
4. Removing songs from the front.
5. Removing songs from the middle.
6. Accessing the k-th song.
7. Iterating through the entire playlist.

Execution time is measured using System.nanoTime() over repeated runs.

4. Complexity Comparison

Operation	ArrayList	LinkedList	ArrayDeque
Add at end	**O(1) amortized**	**O(1)**	**O(1) amortized**
Insert at front	**O(n)**	**O(1)**	**O(1)**
Insert in middle	**O(n)**	**O(n)**	**Not supported**
Remove at front	**O(n)**	**O(1)**	**O(1)**
Remove in middle	**O(n)**	**O(n)**	**Not supported**
Get k-th element	**O(1)**	**O(n)**	**Not supported**
Iterate	**O(n)**	**O(n)**	**O(n)**

5. Space Complexity

- **ArrayList:** Uses a resizable array with one reference slot per allocated capacity position.
- **LinkedList:** Uses linked nodes containing references to the previous and next nodes, along with the element.
- **ArrayDeque:** Uses a resizable circular array to store elements.

ArrayList generally uses less memory per element than LinkedList because it does not require separate linked nodes.

6. Changed Condition – Up Next Queue

The application introduces an Up Next queue where songs are frequently added at both ends and removed from the front.

ArrayDeque is more appropriate because:

- It supports insertion at both ends.
- It supports removal from the front.
- It does not require shifting elements.
- It provides efficient queue and deque operations.

Therefore, ArrayDeque is recommended for the Up Next feature.

7. Alternative Concept – Plain Array

A plain array can store songs in sequence. To insert or remove an element, the remaining elements must be shifted manually.

- Accessing the k-th song takes O(1).
- Adding at the end takes O(1) if space is available.
- Inserting or deleting at the front or middle takes O(n).
- Iteration takes O(n).

A plain array is less flexible because its size is fixed unless resizing is implemented manually.

8. Result

ArrayList is a suitable choice for playlists where users frequently access songs by index, add songs at the end and iterate through the collection.

LinkedList is useful when frequent insertions and removals are performed at known positions or at the ends.

ArrayDeque is the preferred choice for the Up Next queue because it efficiently supports operations at both ends.

Final decision: Use ArrayList for the main music playlist and ArrayDeque for the Up Next queue.

9. Coding

Main.java

Create the Java file inside:

Assignments 02\Task02\Main.java

This program automatically tests all three collection types at the three selected sizes and displays average execution times.

```
import java.util.*;
import java.util.function.*;

public class Main {

    static volatile long result;

    interface ListFactory {
        List<Integer> create(int size);
    }

    interface Action {
        void run(List<Integer> list, int operations);
    }

    interface DequeAction {
        void run(Deque<Integer> deque, int operations);
    }
```

```
static List<Integer> createList(int n, boolean linked) {
    List<Integer> list = linked
            ? new LinkedList<>()
            : new ArrayList<>();

    for (int i = 0; i < n; i++) {
        list.add(i);
    }

    return list;
}

static double measureList(
        ListFactory factory, Action action,
        int n, int operations) {

    long total = 0;

    for (int r = 0; r < 5; r++) {
        List<Integer> list = factory.create(n);

        long start = System.nanoTime();
```

```
        action.run(list, operations);
        total += System.nanoTime() - start;
    }

    return total / 5.0 / 1_000_000.0;
}

static double measureDeque(
        DequeAction action, int n, int operations) {

    long total = 0;

    for (int r = 0; r < 5; r++) {
        Deque<Integer> deque = new ArrayDeque<>();

        for (int i = 0; i < n; i++) {
            deque.addLast(i);
        }

        long start = System.nanoTime();
        action.run(deque, operations);
        total += System.nanoTime() - start;
    }
```

```
    return total / 5.0 / 1_000_000.0;
}
static void test(int n) {
    int operations = Math.min(n, 1000);
    ListFactory array = size -> createList(size, false);
    ListFactory linked = size -> createList(size, true);
    System.out.println("\n================================");
    System.out.println("Playlist Size: " + n);
    System.out.println("================================");

    System.out.printf("%-25s %-12s %-12s%n",
            "Operation", "ArrayList", "LinkedList");

    Action addEnd = (list, ops) -> {
        for (int i = 0; i < ops; i++)
            list.add(i);
    };

    Action addFront = (list, ops) -> {
        for (int i = 0; i < ops; i++)
            list.add(0, i);
    };
```

```
Action addMiddle = (list, ops) -> {
    for (int i = 0; i < ops; i++)
        list.add(list.size() / 2, i);
};

Action removeFront = (list, ops) -> {
    for (int i = 0; i < ops; i++)
        list.remove(0);
};

Action removeMiddle = (list, ops) -> {
    for (int i = 0; i < ops; i++)
        list.remove(list.size() / 2);
};

Action access = (list, ops) -> {
    long sum = 0;
    for (int i = 0; i < 10000; i++)
        sum += list.get(i % list.size());
    result = sum;
};
```

```
Action iterate = (list, ops) -> {
    long sum = 0;
    for (int value : list)
        sum += value;
    result = sum;
};

Action[] actions = {
    addEnd, addFront, addMiddle,
    removeFront, removeMiddle, access, iterate
};

String[] names = {
    "Add at end",
    "Insert at front",
    "Insert in middle",
    "Remove at front",
    "Remove in middle",
    "Access k-th element",
    "Full iteration"
};

for (int i = 0; i < actions.length; i++) {
```

```
    int count = (i == 5) ? 10000 :
                (i == 6) ? 1 : operations;
    double a = measureList(
            array, actions[i], n, count);
    double l = measureList(
            linked, actions[i], n, count);

    System.out.printf("%-25s %-12.3f %-12.3f%n",
            names[i], a, l);
}

System.out.println("\nArrayDeque - Up Next Queue");

double addFirst = measureDeque((deque, ops) -> {
    for (int i = 0; i < ops; i++)
        deque.addFirst(i);
}, n, operations);

double addLast = measureDeque((deque, ops) -> {
    for (int i = 0; i < ops; i++)
        deque.addLast(i);
}, n, operations);
```

```
    double removeFirst = measureDeque((deque, ops) -> {
        for (int i = 0; i < ops; i++)
            deque.removeFirst();
    }, n, operations);

    System.out.printf("Add at front:    %.3f ms%n", addFirst);
    System.out.printf("Add at end:      %.3f ms%n", addLast);
    System.out.printf("Remove at front: %.3f ms%n", removeFirst);

    System.out.println(
            "\nArrayDeque does not support indexed access or middle insertion.");
}

public static void main(String[] args) {

    System.out.println("MUSIC PLAYLIST PERFORMANCE TEST");
    System.out.println("--------------------------------");

    test(1000);
    test(10000);
    test(50000);
```

```
        System.out.println("\nAll experiments completed.");
    }
}
import java.util.*;

public class Main {

    static volatile long result;

    static String buildString(int fields) {
        String ticket = "";

        for (int i = 1; i <= fields; i++) {
            ticket = ticket + "Field" + i + " ";
        }

        return ticket;
    }

    static String buildStringBuilder(int fields) {
        StringBuilder ticket = new StringBuilder();

        for (int i = 1; i <= fields; i++) {
```

```
        ticket.append("Field").append(i).append(" ");
    }

    return ticket.toString();
}

static String buildStringBuffer(int fields) {
    StringBuffer ticket = new StringBuffer();

    for (int i = 1; i <= fields; i++) {
        ticket.append("Field").append(i).append(" ");
    }

    return ticket.toString();
}

static long measure(int method, int tickets, int fields) {
    long start = System.nanoTime();
    long total = 0;

    for (int i = 0; i < tickets; i++) {
        String ticket;
```

```
        if (method == 1)
            ticket = buildString(fields);
        else if (method == 2)
            ticket = buildStringBuilder(fields);
        else
            ticket = buildStringBuffer(fields);

        total += ticket.length();
    }

    result = total;
    return System.nanoTime() - start;
}

static void test(int tickets, int fields) {

    System.out.println("\nTickets: " + tickets +
            " | Fields: " + fields);

    // Warm-up
    for (int i = 0; i < 3; i++) {
        measure(1, tickets, fields);
        measure(2, tickets, fields);
```

```
    measure(3, tickets, fields);
}

String[] names = {
    "String",
    "StringBuilder",
    "StringBuffer"
};

for (int method = 1; method <= 3; method++) {

    long totalTime = 0;

    for (int run = 0; run < 5; run++) {
        totalTime += measure(method, tickets, fields);
    }

    double average =
            totalTime / 5.0 / 1_000_000.0;

    System.out.printf("%-15s : %.3f ms%n",
            names[method - 1], average);
}
```

```
    }

    public static void main(String[] args) {

        System.out.println("RAILWAY E-TICKET PERFORMANCE TEST");
        System.out.println("--------------------------------");

        test(1, 5);
        test(10000, 50);
        test(20000, 100);

        System.out.println("\nPerformance testing completed.");
    }
}
```

Output:

```
MUSIC PLAYLIST PERFORMANCE TEST
-------------------------------
==================================
Playlist Size: 1000
==================================
Operation              ArrayList    LinkedList
Add at end             ...          ...
Insert at front        ...          ...
```

Insert in middle ... ...

Remove at front ... ...

Remove in middle ... ...

Access k-th element ... ...

Full iteration ... ...

ArrayDeque - Up Next Queue

Add at front: ... ms

Add at end: ... ms

Remove at front: ... ms

==========

Playlist Size: 10000

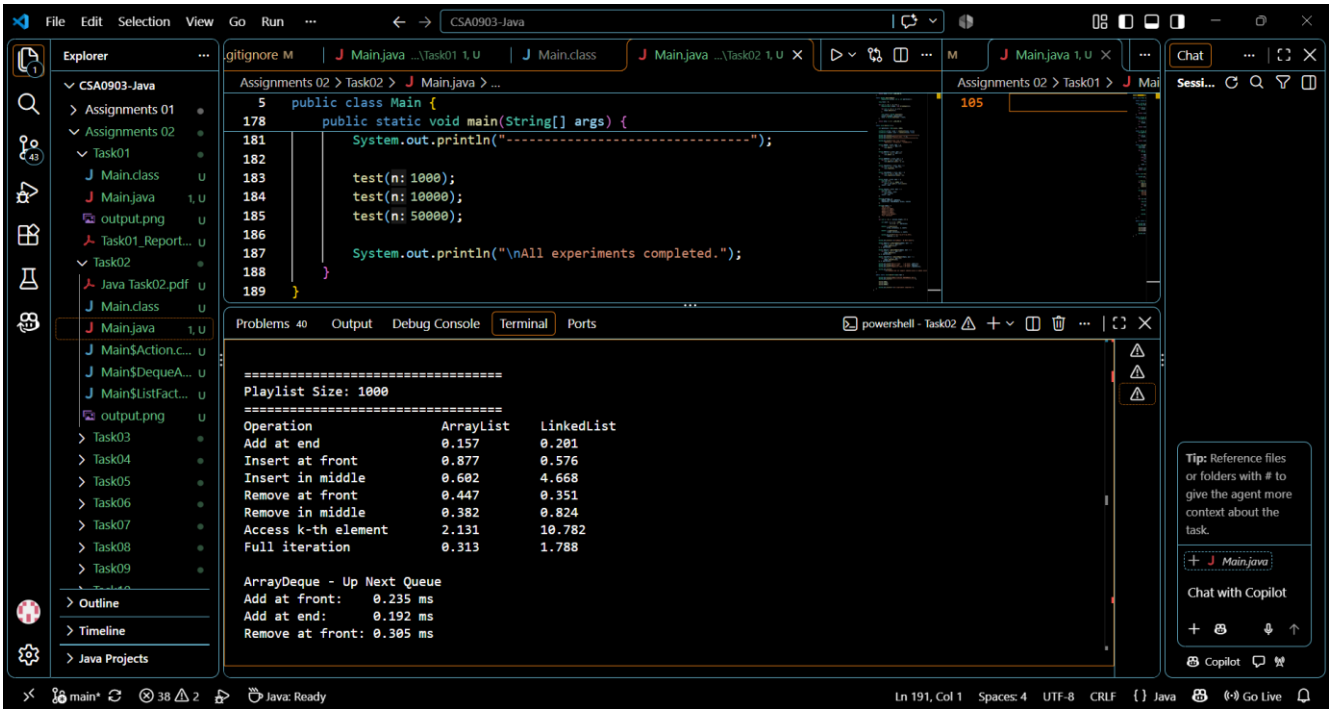
==========

==========

Playlist Size: 50000

==========

All experiments completed.**Implementation:**


File Edit Selection View Go Run
CSA0903-Java
Explorer
CSA0903-Java
Assignments 01
Assignments 02
Task01
Main.class
Main.java
output.png
Task01_Report...
Task02
Java Task02.pdf
Main.class
Main.java
Main$Action.c...
Main$DequeA...
Main$ListFact...
output.png
Task03
Task04
Task05
Task06
Task07
Task08
Task09
Outline
Timeline
Java Projects
.gitignore M
Main.java ...\Task01 1, U
Main.class
Main.java ...\Task02 1, U
Main.java 1, U
Chat
Assignments 02 > Task02 > Main.java > ...
Assignments 02 > Task01 > Mai
5 public class Main {
178 public static void main(String[] args) {
181 System.out.println("--------------------------------");
183 test(n: 1000);
184 test(n: 10000);
185 test(n: 50000);
187 System.out.println("\nAll experiments completed.");
188 }
189 }
105
Problems 40
Output
Debug Console
Terminal
Ports
powershell - Task02
==================================
Playlist Size: 1000
==================================
Operation ArrayList LinkedList
Add at end 0.157 0.201
Insert at front 0.877 0.576
Insert in middle 0.602 4.668
Remove at front 0.447 0.351
Remove in middle 0.382 0.824
Access k-th element 2.131 10.782
Full iteration 0.313 1.788
ArrayDeque - Up Next Queue
Add at front: 0.235 ms
Add at end: 0.192 ms
Remove at front: 0.305 ms
Tip: Reference files or folders with # to give the agent more context about the task.
Main.java
Chat with Copilot
Copilot
main*
38 2
Java: Ready
Ln 191, Col 1
Spaces: 4
UTF-8
CRLF
Java
Go Live